

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

SEMESTER PROJECT

INTELLIGENT MAINTENANCE AND OPERATIONS SYSTEMS (IMOS)

Spatial-Temporal Denoising Diffusion Probabilistic Model for Industrial IoTs

Author

Matya Aydin

Professor

Olga Fink

Supervisor

Keivan Faghieh Niresi



Abstract

Accurate probabilistic load forecasting is critical for maintaining grid stability and optimizing modern energy systems. It remains challenging due to the inherent volatility of energy consumption and the frequent lack of explicit spatial information. This project extends existing diffusion-based models for spatial-temporal graphs framework to address scenarios where the network is unknown. We propose and evaluate two primary enhancements: latent graph structure learning (including dynamic k-Nearest Neighbors (k-NN) and static Data Adaptive Graph Generation (DAGG)) and the integration of alternative spatial and temporal blocks. The model was tested on two energy datasets. We conclude by showing that such probabilistic models can also be used for anomaly detection.

Contents

1	Introduction	3
2	Preliminaries	3
3	Related work	3
3.1	Time series forecasting	3
3.2	Graph neural networks	3
3.3	Spatial-Temporal Graph Neural Networks	4
3.4	Diffusion models	4
3.5	Probabilistic Spatio-Temporal Graph	4
3.6	Graph learning	5
3.6.1	k-NN graph learning	5
3.6.2	Data adaptative graph generation	5
4	Contribution	5
4.1	Graph learning	6
4.2	Spatial block	6
4.3	Temporal block	6
4.4	Anomaly detection	7
5	Experiments	7
5.1	Setup and data preprocessing	7
5.1.1	Datasets	7
5.1.2	Metrics	9
5.1.3	Benchmarks	10
5.1.4	Anomaly detection	10
5.2	Ablation study	10
5.2.1	Graph learning	10
5.2.2	Spatial block	11
5.2.3	Temporal block	11
5.3	Results	12
5.3.1	Forecasting results	12
5.3.2	Impact of horizon	14
5.3.3	Anomaly Detection	14
6	Conclusion	15
7	Appendix	18
7.1	Experimental details	18

1 Introduction

Accurate load forecasting across diverse consumer instances has become a cornerstone of energy systems. The ability to predict demand with high precision is essential for ensuring grid stability, optimizing real-time dispatch, and facilitating dynamic pricing mechanisms that balance supply and demand. However, energy consumption is characterized by extreme volatility driven by human behavior and environmental variable. Consequently, traditional point forecasting often fails to capture the associated risks, necessitating a probabilistic forecasting approach by modeling the full predictive distribution rather than a single scalar value.

Individual consumer instances are inherently interconnected, typically organized as a distributed network. These systems are modeled as spatial-temporal graphs, where spatial prior knowledge such as physical proximity is used to define the relational topology. Leveraging such structural inductive biases allows models to capture the propagation of demand patterns across the network.

Despite the efficacy of graph-based methods, relying exclusively on predefined, static adjacency matrices presents significant challenges. Human-built graphs often reflect anatomical or geographic assumptions that may not align with the latent functional dependencies of energy flow. Furthermore, such priors can introduce confirmation bias, where the model is constrained to learn from non-existent patterns or incomplete domain knowledge. In many practical scenarios, explicit spatial information is entirely absent or commercially restricted, rendering the manual construction of a representative graph mathematically intractable. This necessitates the development of models capable of latent graph structure learning, where the network topology is inferred directly from the temporal dynamics of the data.

2 Preliminaries

Let $G = \{V, E, A\}$ be a graph of V nodes connected by edges in the set E , with relationships encoded in the adjacency matrix $A \in \mathbb{R}^{V \times V}$.

Consider a feature matrix $\mathbf{X}_t \in \mathbb{R}^{F \times V}$ which associates an F -dimensional feature vector $\mathbf{x}_t \in \mathbb{R}^F$ to each node at time $t \in \{1, \dots, T\}$. In this work, we focus on the scalar case and fix $F = 1$ for the remaining parts.

We aim to learn a function $f_\theta(\mathbf{X})$, which, given an history of the past T_h features matrices at time t , will predict the next T_p ones:

$$f_\theta : \{\mathbf{X}_{t-T_h}, \dots, \mathbf{X}_t\} \rightarrow \{\mathbf{X}_{t+1}, \dots, \mathbf{X}_{t+T_p}\}$$

θ are the parameters to be learned from past observations. To make the predictions probabilistic, f_θ contains randomness, encoded as a learned distribution. At each time step t , we predict n_{sampling} times and aggregate the result by a simple average.

3 Related work

3.1 Time series forecasting

As a time series can be modeled as a sequence, standard time series forecasting methods include Long Short Term Memory (LSTM), Gated Recurrent Unit (GRU) and Transformed-based models. While these architectures provide strong results by capturing intrinsic dependencies in individual time series, they do not take into account the underlying graph based structure of the data of interest in this work.

3.2 Graph neural networks

Graph Neural Networks (GNNs) have emerged as a solution to apply deep learning models to graph data [1]. While Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs) focused on the special case of sequence and grid-structured data, GNNs generalize this by adapting the aggregation part to arbitrary geometry by using the adjacency

matrix A .

The fundamental operation in GNNs is message passing, where node representations are iteratively updated by aggregating features from their neighborhood.

GNNs applications range from node-level, edge-level or graph-level classification and regression. In this work, we focus on their ability to forecast each node feature.

3.3 Spatial-Temporal Graph Neural Networks

Spatial-Temporal Graph Neural Networks (STGNN) [2] are a special case of GNNs used to handle applications where the graph structure, or node features, change over time and the graph is based on physical proximity.

They have primarily been used in traffic forecasting and energy forecasting.

3.4 Diffusion models

Denoising Diffusion Probabilistic Models (DDPMs) [3] have emerged as a powerful class of generative models, at first for image generation. Their core mechanism involves a forward diffusion process that systematically transforms data into Gaussian noise through a predefined Markov chain, and a parameterized reverse process that learns a denoiser.

Forward process

The forward process is a Markov chain, which progressively adds noise to an initial observation x_0 :

$$q(x_{1:N}|x_0) = \prod_{n=1}^N q(x_n|x_{n-1}) \quad (1)$$

$q(x_n|x_{n-1}) = \mathcal{N}(x_n; \sqrt{1 - \beta_n}x_{n-1}, \beta_n I)$ is a gaussian distribution. I is the identity matrix and β_n is the noise schedule, built such that β_n slowly increases from 0 to 1.

Reverse process

The reverse process learns to recover x_0 from

pure noise $p(x_N) = \mathcal{N}(x_N; 0, I)$. It is approximated by a Markov chain with learnable gaussian transitions:

$$p_\theta(x_{0:N}) = p(x_N) \prod_{n=N}^1 p_\theta(x_{n-1}|x_n) \quad (2)$$

With $p_\theta(x_{n-1}|x_n) = \mathcal{N}(x_{n-1}; \mu_\theta(x_n, n), \sigma_\theta(x_n, n))$. The parameterization from [3] is:

$$\mu_\theta(x_n, n) = \frac{1}{\alpha_n} \left(x_n - \frac{\beta_n}{\sqrt{1 - \alpha_n}} \epsilon_\theta(x_n, n) \right) \quad (3)$$

$$\sigma_\theta(x_n, n) = \frac{1 - \alpha_{n-1}}{1 - \alpha_n} \beta_n \quad (4)$$

With $\alpha_n = \prod_{i=1}^n (1 - \beta_i)$. Finally, $\epsilon_\theta(x_n, n)$ is a trainable denoiser, whose parameters are learned by iteratively minimizing the following optimization problem:

$$\min_{\theta} \mathbb{E} \|\epsilon - \epsilon_\theta(\sqrt{\alpha_n}x_0 + \sqrt{1 - \alpha_n}\epsilon, n)\|_2^2 \quad (5)$$

Where $\epsilon \sim \mathcal{N}(0, I)$. In the domain of time series forecasting, recent research has transitioned from traditional autoregressive models to diffusion-based frameworks to better capture complex, non-linear dependencies and provide uncertainty quantification [4], [5]. Unlike standard unconditional generation, forecasting is formulated as a conditional diffusion task. The model learns to generate future values by conditioning the reverse process on historical observations.

While these methods have proven effective for both univariate and multivariate datasets, their application to spatial-temporal graph-structured data is limited.

3.5 Probabilistic Spatio-Temporal Graph

Probabilistic Spatio-Temporal Graph (Diff-STG) [6] is the first model to leverage probabilistic forecasting in the STGNN framework by using a diffusion process. An overview of its

architecture is presented in Figure 1.

The denoiser ϵ_θ is a Unet-styled neural networks adapted to spatial-temporal graphs (UGnet). It consists of 5 (2 downsampling, one middle and 2 upsampling) spatial-temporal blocks. Each spatial-temporal block applies two temporal causal convolutions and one graph convolution. The step of the diffusion process is taken into account by adding a positional embedding. The denoising process is conditioned on the graph G and previous node features and aims at modeling the prediction uncertainty, which leads to a conditional adaptation of the framework presented in Section

3.4:

The learned reverse process becomes:

$$p_\theta(\mathbf{X}_{0:N}^{all} | \mathbf{X}_{msk}^{all}, G) = p(\mathbf{X}_N^{all}) \prod_{n=N}^1 p_\theta(\mathbf{X}_{n-1}^{all} | \mathbf{X}_n^{all}, \mathbf{X}_{msk}^{all}, G) \quad (6)$$

The objective becomes

$$\min_{\theta} \mathbb{E}_{\mathbf{X}_0^{all}, \epsilon} \|\epsilon - \epsilon_\theta(\mathbf{X}_n^{all}, n | \mathbf{X}_{msk}^{all}, G)\|_2^2 \quad (7)$$

$\mathbf{X}^{all}$ concatenates the historical features and features to be predicted: $\mathbf{X}^{all} = [\mathbf{X}^h; \mathbf{X}^p]$. $\mathbf{X}_{msk}^{all}$ represents the history by masking all future time steps in $\mathbf{X}^{all}$.

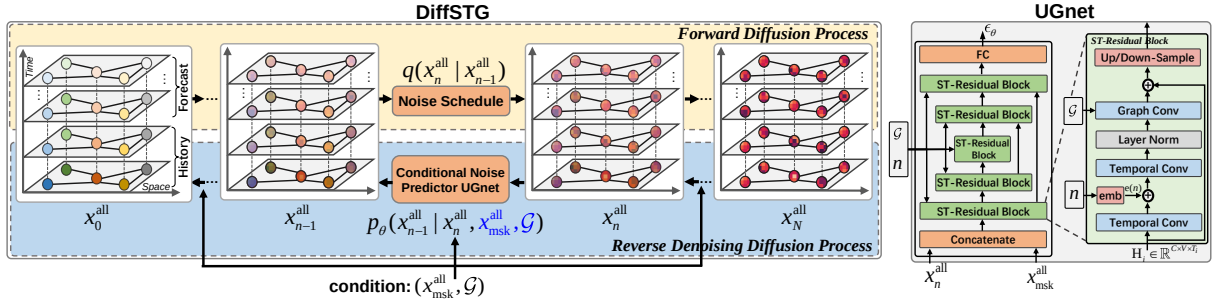


Figure 1: DiffSTG architecture

3.6 Graph learning

GNNs and STGNNs rely on the knowledge of a graph, which may or may not be time-varying. Recent advancements allow the underlying structure to be learned directly from data.

3.6.1 k-NN graph learning

As in [7], we can learn a softmax-parameterized distribution over a k-NN graph by assigning a score ϕ_{ij} to each edge. Namely, for each node V_i , we sample K neighbors without replacement from a learnable matrix $\Phi \in \mathbb{R}^{V \times V}$:

$$\{\text{Neighb}(V_i)_k\}_{k=1}^K \sim \Phi_i \quad (8)$$

$$\text{Where } \Phi_{ik} = \frac{\exp \phi_{ik}}{\sum_{j=1}^N \exp \phi_{ij}}.$$

3.6.2 Data adaptative graph generation

Data Adaptative Graph Generation (DAGG) [8] adds node embeddings $E_A \in \mathbb{R}^{V \times h}$, where h is the hidden size, to learn the normalized adjacency matrix:

$$D^{-\frac{1}{2}} A D^{-\frac{1}{2}} = \text{softmax}(\text{ReLU}(E_A E_A^T)) \quad (9)$$

4 Contribution

This work aims at extending diffusion-based models for spatial-temporal graphs, such as the work presented in Section 3.5. First we assess how to tackle the problem of not having access to a graph, instead it is learned. After that, we study the impact of changing the operators that handle spatial and temporal dependencies. Finally, we show that, apart from forecasting, due to its probabilistic nature, the model can

also be used to perform anomaly detection.

4.1 Graph learning

As the framework presented in Section 3.5 relies on a graph, which can be missing in some applications, we propose to add graph imputation in the model. We assess both dynamic and static graph learning. Dynamic graph learning, presented in Section 3.6.2, samples a new graph for each batch while static graph learning, presented in Section 3.6.1, learns one graph for all batches.

4.2 Spatial block

The work done on diffusion-based models for spatial-temporal graphs only uses the standard graph convolution in the spatial block. We try to replace it with a diffusion convolution (Diffconv) [9], [10] and Graph Attention Networks (GATConv) [11]. Diffusion convolution captures spatial dependency using bidirectional random walks on the graph.

GATConv utilizes masked self-attention layers to assign learned importance weights to neighbors. A visualization of each graph convolution is presented on Figure 2.

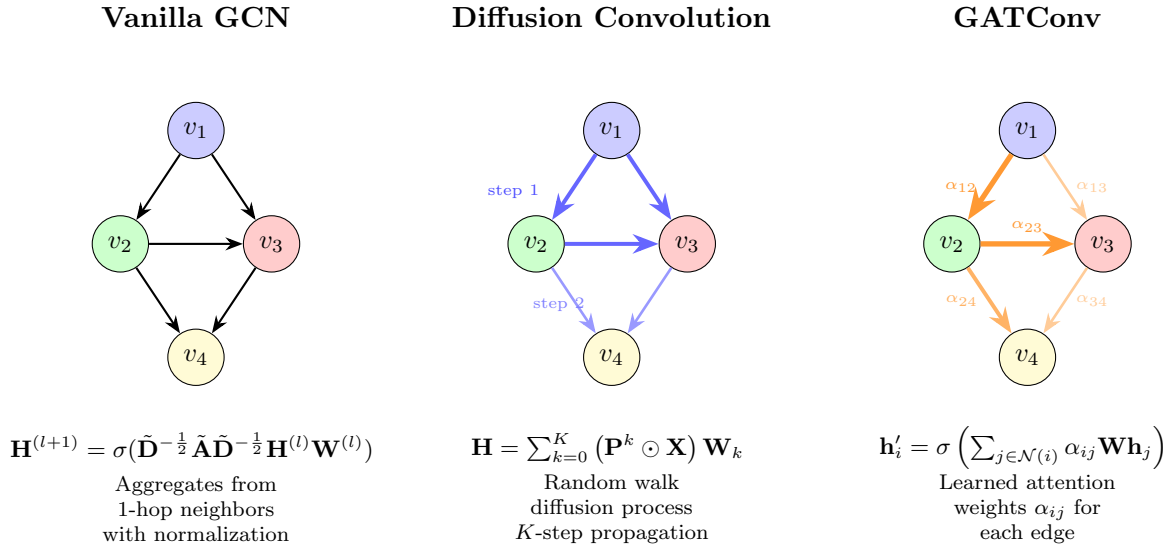


Figure 2: Comparison of graph convolutions:

Vanilla GCN (left) uses spectral-based normalization, $\tilde{\mathbf{D}}$ denotes the degree matrix, $\tilde{\mathbf{A}}$ denotes the adjacency matrix, $\mathbf{W}^{(l)}$ is the shared weight matrix at layer l and $\sigma(\cdot)$ is an activation function.

Diffusion convolution (center) employs a diffusion-based random walk. $\mathbf{P}$ represents the transition matrix.

GAT (right) utilizes dynamic attention mechanisms. $\mathcal{N}(i)$ denotes the neighborhood of node i .

4.3 Temporal block

The work done on diffusion-based models for spatial-temporal graphs only assessed gated causal convolution in the temporal block. We now study the impact of replacing this block by either GRU, LSTM or Transformer blocks,

as they have intensively been used for time series forecasting, after showing success in causal language modeling [12], [13]. While LSTM and GRU utilize recurrent gating mechanisms to capture long-range temporal dependencies through internal memory states, Transformers employ multi-head self-attention to model

global correlations across the entire time series sequence simultaneously.

4.4 Anomaly detection

By using probabilistic forecasting and spatial information, we show that the work can also be used to detect anomalies, which are samples that differ from the expected behaviour in each time series. They can appear due to measurement error, outages or rare events.

Anomalies are identified through a Z-score thresholding method based on the probabilistic predictions of the model. A sample y_t is classified as an anomaly if:

$$\frac{|\bar{y}_t - y_t|}{\sigma_t} > k \quad (10)$$

In this expression, $\bar{y}_t$ and σ_t represent the empirical mean and standard deviation, respectively. These values are derived from n_{sample} stochastic forward passes at each timestep. The constant k determines the sensitivity of the detection. A lower k value increases the detection rate, whereas a higher k value reduces it. This detection process is independent of the time index. It does not rely on running statistics calculated over a sliding temporal window. This approach offers robust detection for initial observations where rolling statistics are often noisy. Running statistics are also susceptible to corruption during prolonged sensor failures. Persistent anomalies or zero-value periods can bias the running mean and variance. Such biases frequently lead to the misclassification of future samples.

Results are presented in Section 5.3.3.

5 Experiments

For reproducibility, experimental details are shared in Section 7.1.

5.1 Setup and data preprocessing

5.1.1 Datasets

To assess our results, we apply our model to two different energy forecasting datasets:

- **EWZ Dataset:** Heating consumption measurements from sensors located in Zurich from 2016 to 2025.
- **Elergone:** Electricity load dataset based on measurements in Portugal from 2011 to 2014.

As spatial data was lacking for the EWZ dataset, when we do not learn the graph, we create a binarized adjacency matrix A based on temporal correlation between sensors:

$$A_{ij} = \begin{cases} 1 & \text{if } \rho(X_i, X_j) \geq \tau \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

Where $\rho(\cdot, \cdot)$ is the correlation operator. From this definition, it is easy to see that $A_{ii} = 1 \forall i$, a key component in graph convolution layer. τ has been chosen such that A is 80% sparse by using quantiles. This choice is motivated by the fact that most real graphs are sparse. Figure 3 shows the resulting adjacency matrices.

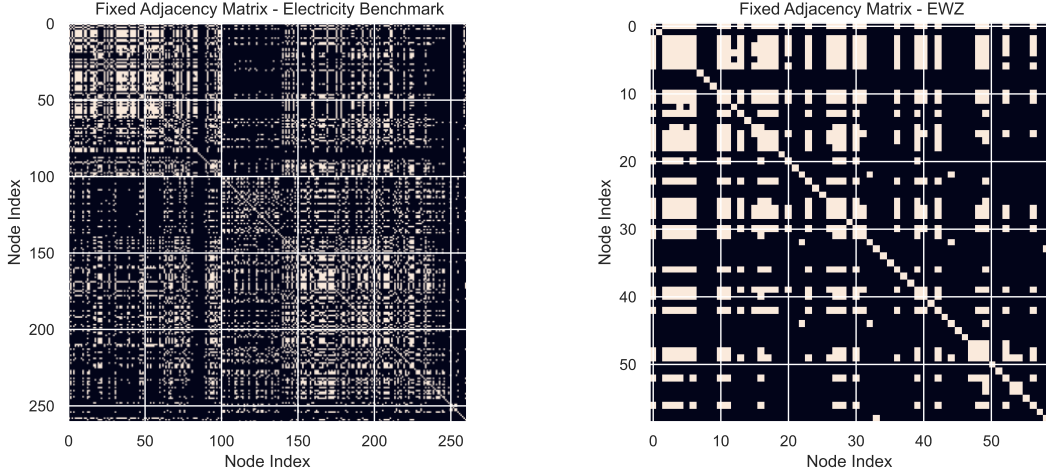


Figure 3: Adjacency matrix for each dataset

For the graph learning, as each node is only connected to the k sampled nodes, the sparsity rate κ is given by

$$\kappa(k) = 1 - \frac{k}{V} \quad (12)$$

Table 1 presents statistics for each dataset before any preprocessing.

Dataset	V	Period	T
EWZ	102	15 minutes	340462
Elergone	370	15 minutes	140256

Table 1: Statistics before preprocessing

To make the data more suitable for our forecasting task, the following steps have been applied:

- **Downsampling:** To make each time series less noisy, the sampling rate has been changed from 15 minutes to 24 hours for EWZ and from 15 minutes to one hour for Elergone, by summing contiguous measurements.
- **Temporal intersection:** To address time series samples where some nodes show constant zero values, which could be due to

the fact that the sensor was either not yet operational or non-existent, we use a cut-off index across all time series, ensuring the analysis begins when all relevant nodes start providing non-zero data.

- **Z-score Standardization:** We subtract mean from each node’s time series and divide it by its standard deviation.

Finally, as total number of initial nodes was low, a visual inspection of the nodes has been done to keep only the nodes where the time series had expected properties: Periodicity and maximal values during the cold months.

Table 2 presents statistics for each dataset after preprocessing. Figures 4 and 5 display the time series of some selected remaining nodes before Z-score standardization. Table 2 presents statistics after preprocessing. From these new sampling rates, we will consider 1 week history, next day horizon for EWZ and one day history, next hour horizon for Elergone.

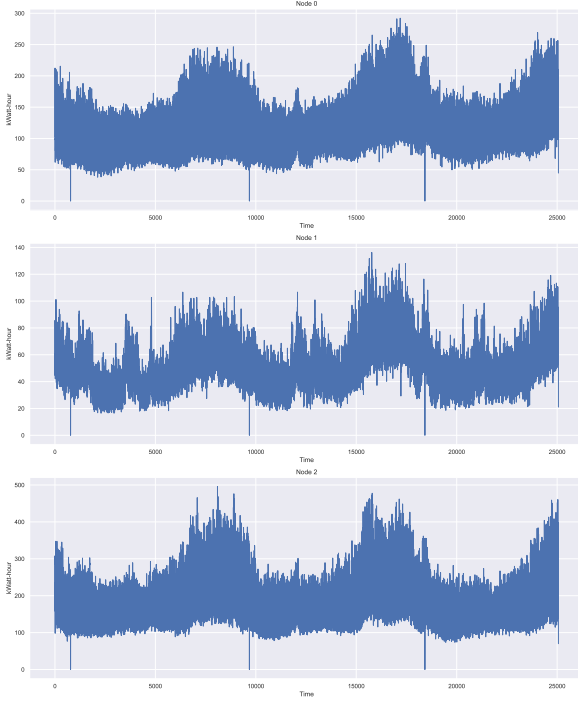


Figure 4: Sampled Elergone time series

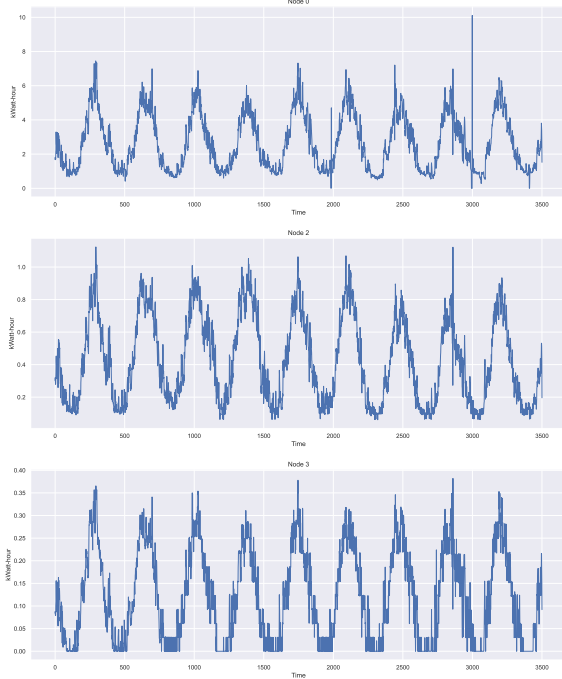


Figure 5: Sampled EWZ time series

Dataset	V	Period	T
EWZ	59	24 hours	3501
Elergone	261	1 hour	25065

Table 2: Statistics after preprocessing

5.1.2 Metrics

Standard regression metrics are applied:

1. Root Mean Square Error (RMSE):

$$\text{RMSE}(\hat{y}, y) = \sqrt{\frac{1}{T} \sum_{i=1}^T (\hat{y}_i - y_i)^2} \quad (13)$$

2. Mean absolute error (MAE):

$$\text{MAE}(\hat{y}, y) = \frac{1}{T} \sum_{i=1}^T |\hat{y}_i - y_i| \quad (14)$$

We also use Continuous Ranked Probability Score (CRPS), a probabilistic metric:

$$\text{CRPS}(F, x) = \int_{\mathbb{R}} (F(z) - \mathbb{1}_{x \leq z})^2 dz \quad (15)$$

Where F is the estimated probability distribution.

For Anomaly detection, we use accuracy and F1-score:

$$\text{Accuracy} = \frac{TP + TN}{T} \quad (16)$$

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (17)$$

With $\text{Precision} = \frac{TP}{TP+FP}$, $\text{Recall} = \frac{TP}{TP+FN}$ Where TP denotes the number of True Positives, TN the number of True Negatives, FP the number of False Positives and FN the number of False Negatives. By definition, $TP + TN + FP + FN = T$, the total number of time steps.

We apply these metrics to each node's time series individually. The presented results are aggregated over all nodes.

5.1.3 Benchmarks

We compare our work against two other state of the art time series forecasting and imputation models:

1. **Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation (CSDI)** [5], which uses diffusion based on observed data.
2. **Chronos-2** [14], a pretrained foundation model for univariate and multivariate time series forecasting that uses a group attention mechanism.

5.1.4 Anomaly detection

To get ground truth indexes labeled as anomalies, we generate two types of fault presented in [15]:

Drift fault: An additive bias is added to the uncorrupted observation $x(t)$. It grows linearly during M steps, starting at index m and stays at a constant value b for the next K steps:

$$\tilde{x}(t) = \begin{cases} x(t) + \frac{b(t-m+1)}{M}, & 0 \leq t-m < M \\ x(t) + b, & M \leq t-m < M+K \\ x(t), & \text{otherwise} \end{cases}$$

Noise fault: Zero-mean white gaussian noise is added to observations for M consecutive samples, starting at index m :

$$\tilde{x}(t) = \begin{cases} x(t) + w(t), & 0 \leq t-m < M \\ x(t), & \text{otherwise} \end{cases}$$

Where $w(t) \sim \mathcal{N}(0, c)$.

These anomalies are only assessed on the EWZ dataset on 10 nodes. We add one anomaly per node in the normalized test set, for a total of five drift faults and five noise faults. m is sampled uniformly in the corresponding range of indexes. For drift fault, we take $M = 4$ and $K = 3$. For noise fault, we

take $M = 7$. b and c are also uniformly sampled between 1.5 and 2.5 and multiplied by the time series standard deviation to account for the intrinsic volatility of each time series.

5.2 Ablation study

This section presents the results of all adaptations discussed in Section 4. In each table, lower value means better. The best result is in bold.

5.2.1 Graph learning

Tables 3 and 4 present results for all graph learning methods. k is adapted to achieve the desired sparsity rate using Equation 12. A limitation of k-NN graph is its inability to adapt the number of neighbors of each node, which motivates using sparser graph and can explain worse results than DAGG for Elergone dataset. However, k-NN 0.8 sparse graph outperforms DAGG for EWZ dataset, which can be explained by the fact that learning the whole graph at once for a small dataset can lead to overfitting. Having dynamic graph is also less suited for such application where the temporal dependency is in the feature and not in the graph, which should be fixed. Having the best sparsity rate $\kappa^* = 0.8$ for EWZ and $\kappa^* = 0.9$ for Elergone can be explained by the fact that the larger a graph is, the higher the probability of it being sparse is.

Figure 6 shows the adjacency matrix learned by DAGG for elergone dataset before applying softmax as in Equation 9. Some limited clusters are formed.

Graph Method	RMSE	MAE	CRPS
Base	1.308	0.161	0.154
k-NN ($\kappa = 0.75$)	0.738	0.113	0.116
k-NN ($\kappa = 0.8$)	0.688	0.106	0.107
k-NN ($\kappa = 0.85$)	0.914	0.137	0.142
k-NN ($\kappa = 0.9$)	0.731	0.112	0.113
k-NN ($\kappa = 0.95$)	0.927	0.145	0.145
DAGG	0.706	0.119	0.115

Table 3: EWZ graph learning results

Graph Method	RMSE	MAE	CRPS
Base	632	74	0.08
k-NN ($\kappa = 0.75$)	647	80	0.086
k-NN ($\kappa = 0.8$)	866	108	0.12
k-NN ($\kappa = 0.85$)	413	48	0.05
k-NN ($\kappa = 0.9$)	372	46	0.046
k-NN ($\kappa = 0.95$)	1038	113	0.11
DAGG	336	39	0.04

Table 4: Elergone graph learning results

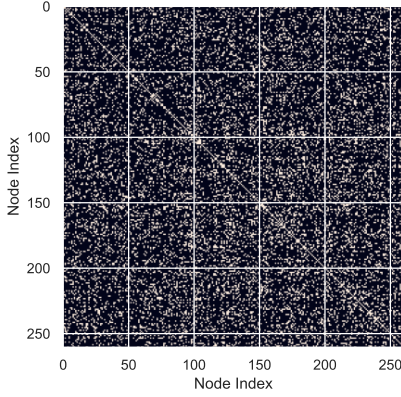


Figure 6: DAGG for Elergone dataset

5.2.2 Spatial block

Poor results for GATConv for EWZ can be explained by a lack of data, as the attention mechanism has a lot of parameters, which requires a substantial amount of data. Without learning the correct weights, the resulting weighted average can be noisy.

Diffconv achieving the best results can be due to the inability of the correlation-based graph to capture the correct spatial neighbors, which can be reached with Diffconv’s random walk process.

Better results for GATConv for Elergone dataset can be explained by (i), having access to an underlying true graph, we can learn to use results from direct neighbors rather than with a random walk and (ii), having more samples allows to correctly learn attention over the nodes.

Block	RMSE	MAE	CRPS
Base	1.308	0.161	0.154
Diffconv	0.629	0.101	0.101
GATConv	2.335	0.216	0.185

Table 5: EWZ spatial block results

Block	RMSE	MAE	CRPS
Base	632	74	0.08
Diffconv	552	67	0.068
GATConv	336	42	0.044

Table 6: Elergone spatial block results

5.2.3 Temporal block

Tables 7 and 8 present the results of changing the temporal block. GRU gets the lowest error for the EWZ dataset, which can be due to the limited amount of data as GRU has less parameters than Transformer and LSTM.

LSTM getting the best results on the Elergone dataset can be due to the larger history length of the one day history, next hour horizon task.

Block	RMSE	MAE	CRPS
Base	1.308	0.161	0.154
LSTM	0.971	0.117	0.106
GRU	0.679	0.111	0.106
Transformer	0.69	0.112	0.115

Table 7: EWZ temporal block results

Block	RMSE	MAE	CRPS
Base	632	74	0.08
LSTM	329	40	0.041
GRU	347	42	0.043
Transformer	415	51	0.052

Table 8: Elergone temporal block results

5.3 Results

5.3.1 Forecasting results

For each component assessed in Section 5.2, we report the results of the best change and compare them. Metrics are based on inverse-transformed data. The second best results are underlined. The best extension for the EWZ dataset is Diffconv. The best extension for Elergone is DAGG. Figures 7 and 8 illustrate the probabilistic forecasting of these models. However, CSDI beats the best extension of the model except for RMSE on the EWZ dataset.

Model	RMSE	MAE	CRPS
Base	1.308	0.161	0.154
k-NN ($\kappa = 0.8$)	0.688	0.106	0.107
Diffconv	0.629	<u>0.101</u>	<u>0.101</u>
GRU	<u>0.679</u>	0.111	0.106
CSDI	0.68	0.1	0.095
Chronos	0.85	0.15	0.14

Table 9: EWZ forecasting results

Model	RMSE	MAE	CRPS
Base	632	74	0.08
DAGG	336	<u>39</u>	<u>0.04</u>
GATConv	336	42	0.044
LSTM	<u>329</u>	40	0.041
CSDI	304	34	0.035
Chronos	1613	149	0.18

Table 10: Elergone forecasting results

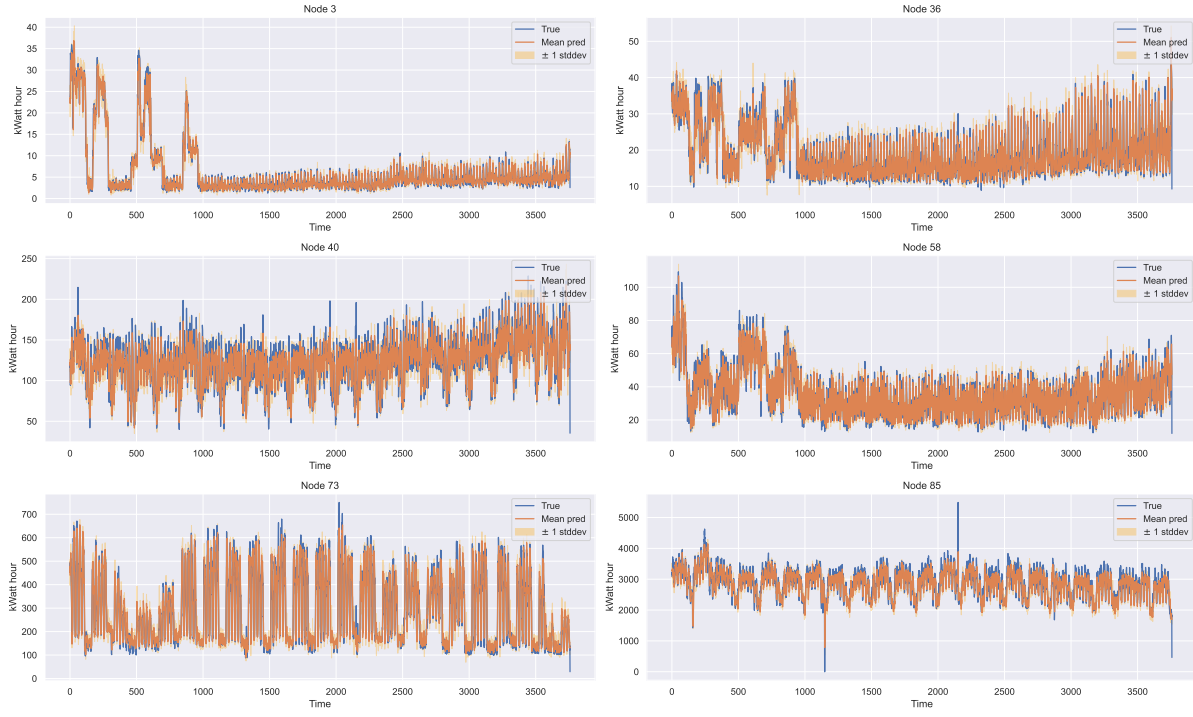


Figure 7: Elergone predictions with the best model

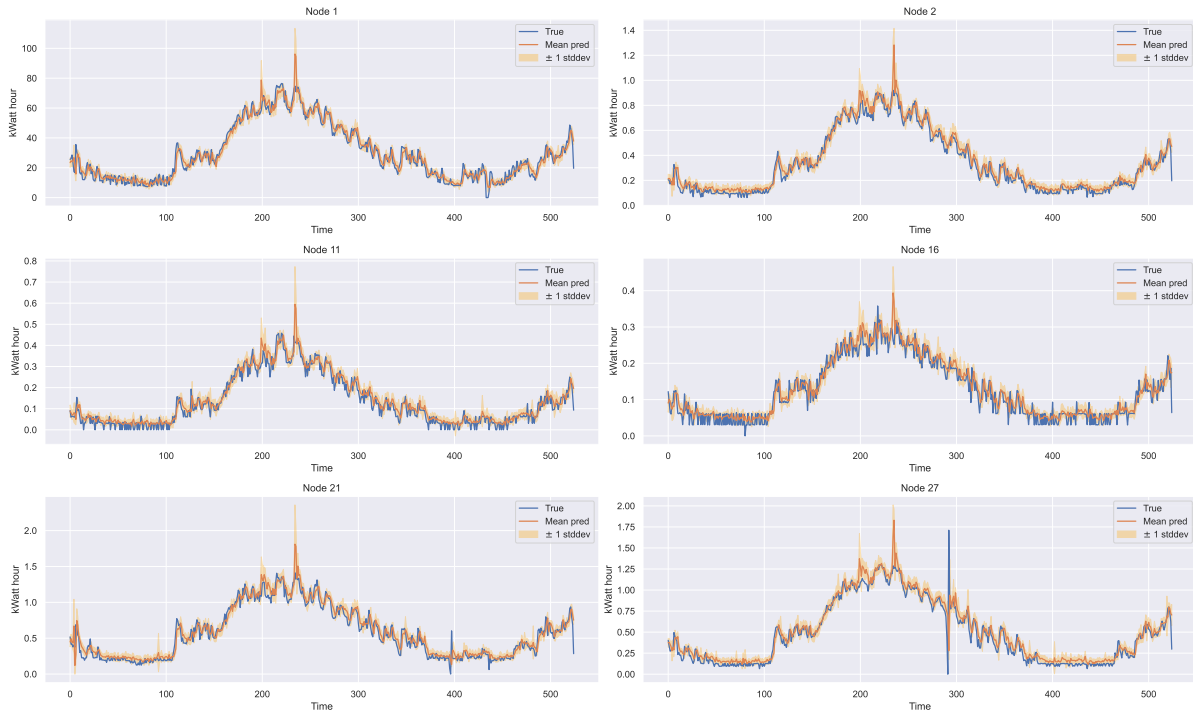


Figure 8: EWZ predictions with the best model

5.3.2 Impact of horizon

We test the results of the best model for horizon larger than one step ahead using only the model resulting from the best change, that is, Diffconv for EWZ and DAGG for Elergone. The slow increase in error for larger horizons can be explained by the fact that DiffSTG is not an autoregressive model and generates predictions for all steps at once.

Horizon	RMSE	MAE	CRPS
1 day	0.629	0.101	0.101
2 days	1.2	0.16	0.16
3 days	1.21	0.15	0.15

Table 11: EWZ larger horizon results

Horizon	RMSE	MAE	CRPS
1 hour	336	39	0.04
2 hours	372	44	0.045
3 hours	382	45	0.047

Table 12: Elergone larger horizon results

5.3.3 Anomaly Detection

This section presents results of the experimental details presented in Section 5.1.4. It is important to note that the values chosen are such that we only have approximately 1.33% synthetic anomalies.

Figure 9 shows Precision and F1-score for various thresholds k . We compare the base model with the best performing model on regression tasks (Diffconv). As anomalies are rare, Accuracy is consistently high across all thresholds. Larger values of k reduce False Positives, thereby increasing precision. This relationship is confirmed by the Precision-Recall curve of the best model presented in Figure 10, where a substantial increase in Precision is observed while Recall remains relatively stable. It is important to note that the dataset inherently contains unlabeled points exhibiting anomalous characteristics beyond the synthetically injected anomalies. These naturally occur-

ring anomalies contribute to the elevated False Positive rate, as the model correctly identifies them as anomalous despite their absence from the ground truth labels.

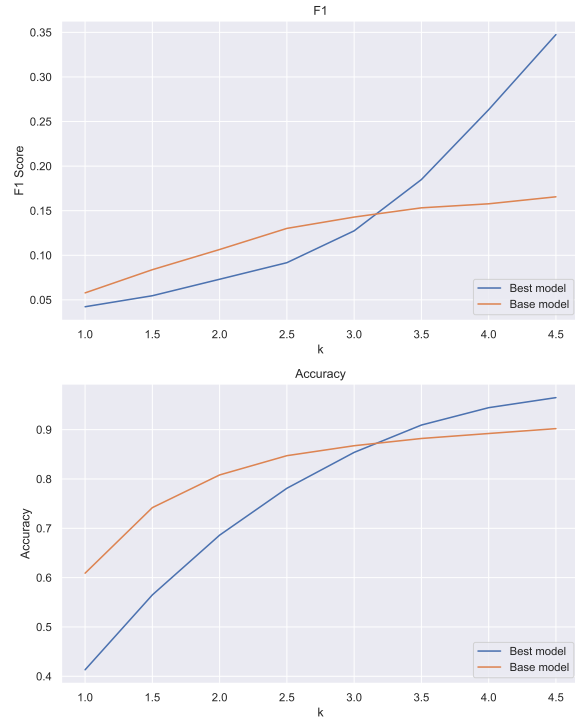


Figure 9: Anomaly detection metrics

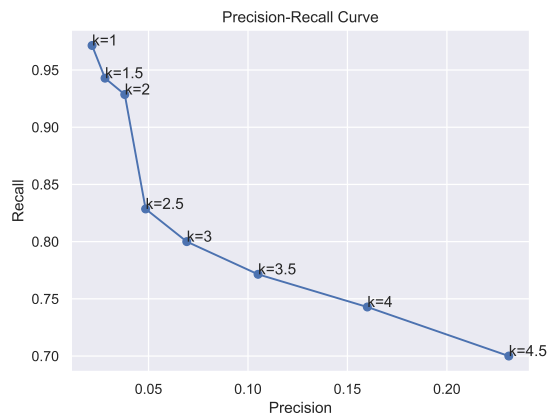


Figure 10: Precision-Recall curve for Diffconv model

6 Conclusion

While the base model is outperformed in most cases, consistently learning a correct graph representation remains challenging and can struggle to be competitive with better spatial and temporal blocks and models that do not assume an underlying graph.

Future work could include making the learned

number of neighbors of each node adaptative, for instance by taking inspiration from nucleus sampling in natural language processing, generating a new graph alongside the diffusion process or carefully mixing up the blocks that provided meaningful increase in performance in order to achieve better performance than CSDI. For anomaly detection, one could assess methods specific to diffusion models and graphs.

References

- [1] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, X. Liu, and L. Sun, “Graph Neural Networks: A Review of Methods and Applications,” *arXiv preprint arXiv:1812.08434*, 2020. [Online]. Available: <https://arxiv.org/abs/1812.08434>
- [2] Y. Sun, H. Lin, M. Wang, X. Wu, S. Tang, and B. Du, “Spatio-Temporal Graph Neural Networks: A Survey,” *arXiv preprint arXiv:2301.10569*, 2023. [Online]. Available: <https://arxiv.org/abs/2301.10569>
- [3] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf>
- [4] Y. Sun, J. Zhang, Q. Lin, H. Liu, H. Zhai, C. Zhao, J. Chen, X. Xu, J. Cao, and Y. Sun, “The Rise of Diffusion Models in Time-Series Forecasting,” *arXiv preprint arXiv:2401.03006*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.03006>
- [5] Y. Tashiro, J. Song, Y. Song, and S. Ermon, “Csdi: Conditional score-based diffusion models for probabilistic time series imputation,” in *Advances in Neural Information Processing Systems*, 2021.
- [6] H. Wen, Y. Lin, Y. Xia, H. Wan, Q. Wen, R. Zimmermann, and Y. Liang, “DiffSTG: Probabilistic spatio-temporal graph forecasting with denoising diffusion models,” in *the 31st ACM International Conference on Advances in Geographic Information Systems*, 2023.
- [7] A. Cini, A. Jenkins, D. Mandic, C. Alippi, and F. M. Bianchi, “Relational Conformal Prediction for Correlated Time Series,” *International Conference on Machine Learning*, 2025.
- [8] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, “Adaptive Graph Convolutional Recurrent Network for Traffic Forecasting,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 17 804–17 815. [Online]. Available: <https://arxiv.org/abs/2007.02842>
- [9] Y. Li, R. Yu, C. Shahidepour, and Y. Tang, “Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting,” *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1707.01926>
- [10] J. Gasteiger, S. Weißenberger, and S. Günnemann, “Diffusion Improves Graph Learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [12] B. Lindemann, T. Müller, H. Vietz, N. Jazdi, and M. Weyrich, “A survey on long short-term memory networks for time series prediction,” *Procedia CIRP*, vol. 99, pp. 650–655, 2021. [Online]. Available: <https://doi.org/10.1016/j.procir.2021.03.088>

- [13] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://openreview.net/forum?id=Jbdc0vTOcol>
- [14] A. F. Ansari, O. Shchur, J. Küken, A. Auer, B. Han, P. Mercado, S. S. Rangapuram, H. Shen, L. Stella, X. Zhang, M. Goswami, S. Kapoor, D. C. Maddix, P. Gueron, T. Hu, J. Yin, N. Erickson, P. M. Desai, H. Wang, H. Rangwala, G. Karypis, Y. Wang, and M. Bohlke-Schneider, “Chronos-2: From univariate to universal forecasting,” *arXiv preprint arXiv:2510.15821*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.15821>
- [15] H. Darvishi, D. Ciunzo, and P. S. Rossi, “A Machine-Learning Architecture for Sensor Fault Detection, Isolation, and Accommodation in Digital Twins,” 2021.

7 Appendix

7.1 Experimental details

In this section, we provide the specific configuration and hyperparameters used to ensure the reproducibility of our results.

Models were trained for a maximum of 100 epochs with early stopping based on metrics measured on the validation set. We used Adam optimizer with learning rate of $\gamma = 2 \times 10^{-3}$ and `ReduceLROnPlateau` scheduler, which reduces the learning rate when the validation loss plateaus. The specific hyperparameter settings are summarized in Table 13.

For EWZ, we set $T_h = 7$, $T_p = 1$ and $n_{\text{sample}} = 16$. For Elergone, we set $T_h = 24$, $T_p = 1$ and $n_{\text{sample}} = 8$. We use n_{sample} at test time but use $\frac{n_{\text{sample}}}{4}$ at validation time.

Table 13: Model and Training Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning Rate (γ)	2×10^{-3}
LR Scheduler	<code>ReduceLROnPlateau</code>
Hidden Layer Size	32
Max Epochs	100
Data Split (Train/Val/Test)	0.70 / 0.15 / 0.15